WEALTHOS : AN AGENTIC AI BASED AUTONOMOUS MULTI-ASSET PORTFOLIO REBALANCING PLATFORM

D. Koushik
Dept. of Cyber Security
Malla Reddy University, Hyderabad
2311cs040045@mallareddyuniversity.ac.in

K. Nandiswar Goud
Dept. of Cyber Security
Malla Reddy University, Hyderabad
2311cs040073@mallareddyuniversity.ac.in

K. Jyothi Priya
Dept. of Cyber Security
Malla Reddy University, Hyderabad
2311cs040076@mallareddyuniversity.ac.in

Dr. G. Latha (Faculty Project Supervisor)
Associate Professor, Department of Cyber Security
Malla Reddy University, Hyderabad
latha.g@mallareddyuniversity.ac.in

Dr. G. Anand Kumar (Dean & Professor)
Dean, Department of Cyber Security
Malla Reddy University, Hyderabad
anandkumar.g@mallareddyuniversity.ac.in

ABSTRACT

When ordinary investors divide their savings into different investments like shares, gold, mutual funds, and crypto, keeping that balance over time is surprisingly difficult. Asset prices change every day at different speeds, and crypto markets stay open all night while stock markets close in the afternoon. When an asset jumps in price, it quietly takes up a much bigger slice of the person's total savings, leaving them exposed to unexpected losses if that market turns. Fixing this balance manually takes continuous monitoring, math calculations, and emotional discipline. Commercial automated advisory apps help, but they charge high yearly fees (often 0.25% to 0.75% of total wealth), check portfolios only once a month or quarter, and require users to upload their private bank and demat details to cloud servers. In this paper, we present WealthOS, a web application that handles multi-asset portfolio rebalancing automatically inside the user's browser without sending any financial records to external servers. Built using React 19, TypeScript, and Tailwind CSS, WealthOS tracks 16 assets across Indian stocks (NSE), gold and silver (MCX), mutual funds (MFAPI), and crypto coins (CoinGecko). The app features a price simulator that refreshes quotes every 3 seconds, a background checker that evaluates portfolio drift every 25 seconds against a 0.4% safety limit, and a cash-first buying engine that uses spare cash to top up lagging assets so investors avoid selling winning assets and paying extra tax. For ease of use, WealthOS includes a voice-enabled AI assistant powered by Google Gemini and local Ollama that explains every trade in plain, everyday language. All trades are saved in a private local ledger inside the browser with fingerprint security and UPI QR codes for simulated deposits. In a 30-day continuous simulation test (43,200 price ticks), WealthOS cut down portfolio drift by 85.6%, boosted the Sharpe ratio from 1.14 to 1.82, and ran drift calculations in just 0.0084 milliseconds. Our project proves that investors can keep their portfolios balanced automatically in their browser without paying advisory fees or compromising their privacy.

Keywords— *Portfolio Rebalancing, Agentic AI, Autonomous Wealth Management, Real-Time Market Feeds, Modern Portfolio Theory, Drift Index, Client-Side Privacy, Gemini AI, Edge Computing.*

1 Introduction

When people start saving and investing money, the most common advice they receive is not to put all their eggs in one basket. Investors generally spread their savings across several different asset types—such as shares in companies, physical or sovereign gold, mutual fund units, and digital crypto assets. At the start, the person picks a clear plan: for instance, holding 28% in equities, 25% in metals, 20% in mutual funds, and 27% in crypto. Spreading money like this protects the investor because when one market falls, another market often stays steady or rises. However, keeping this target mix intact over time is a major challenge in real life because market prices never sit still. As prices rise and fall, the real share of each investment drifts away from the initial plan.

This drift happens rapidly today because different markets run on completely different schedules and move at different speeds. Cryptocurrencies trade around the clock every single day, with sharp price jumps happening late at night or over weekends. Indian equities on the National Stock Exchange (NSE) trade only on weekdays from 9:15 AM to 3:30 PM, changing in value based on corporate quarterly earnings, economic news, and institutional buying. Gold and silver bullion trade on the Multi Commodity Exchange (MCX) from morning until 11:30 PM, acting as safe havens during geopolitical tensions. Meanwhile, mutual funds publish updated values only once per day, late in the evening. Because these markets operate on different clocks, an investor's portfolio balance gets thrown off very quickly.

When one investment experiences a major rally, it takes up a far larger proportion of the investor's total savings than planned. For example, if crypto prices double, crypto might suddenly make up 45% of the person's money instead of 27%. This leaves the person carrying far more risk than they intended, meaning a sudden market pullback can wipe out a huge chunk of their savings. At the same time, investments that did not move or fell in price end up making up a smaller percentage, so the investor misses the chance to buy more of them at lower prices. The remedy is portfolio rebalancing: bringing the weights back to the target percentages so risk stays under control.

Fixing this balance manually is tedious and stressful. A retail investor has to open multiple trading apps, calculate percentages in Excel, and place multiple buy and sell orders. Human emotions make it even harder: people get greedy and hate selling their winning investments, and fear keeps them from putting fresh money into falling assets. Commercial automated advisory apps attempt to solve this, but they charge yearly fees (usually 0.25% to 0.75% of total portfolio value), rebalance only once a month or quarter, and require users to link their private bank accounts and tax IDs to external cloud servers.

To solve these practical issues, we built WealthOS, a free web app that runs in the browser and rebalances multi-asset portfolios automatically. WealthOS monitors 16 assets across stocks, bullion, funds, and crypto using live API endpoints. It includes a price simulator that updates values every 3 seconds, a background check that runs every 25 seconds to catch drift greater than 0.4%, an order creator that uses spare cash first so winning assets are not sold needlessly, a local ledger with fingerprint authentication and UPI QR code support, and a conversational AI assistant using Gemini and local Ollama. Because all data stays in the browser, WealthOS provides automatic rebalancing while keeping user finances completely private.

2 Motivation and Limitations of Existing Systems

Most retail investors manage their investments either by logging into separate broker apps (like Zerodha, Groww, or Upstox) or by signing up for automated robo-advisors. When studying how people manage multi-asset investments today, we observed four clear problems in current tools:

- **Infrequent Rebalancing Checks:** Most automated advisory apps review portfolio percentages only once a month or once a quarter. In fast-moving markets like crypto, an asset can surge or drop by 10% to 15% in just a few days. Waiting a whole month means the investor remains exposed to unintended risk for weeks before anything is fixed.
- **Scattered Investment Accounts:** Investors hold their assets in scattered places: stocks in a demat account, gold in physical form or sovereign bonds, mutual funds on registrar portals, and crypto on specialized exchanges. No simple tool adds up all these assets in real time to show true net worth and rebalance across all of them together.
- **Privacy Risks of Storing Data in Cloud:** Cloud-based wealth managers ask users to connect their bank accounts, PAN numbers, and trading histories to remote company servers. If that company's database is hacked or leaked, the user's private financial data is compromised.
- **No Explanations for Automated Trades:** Most automated tools buy and sell assets silently behind the scenes without explaining the logic to the investor. When users cannot see why a trade was made, they feel uncomfortable trusting the automated system.

Furthermore, ordinary rebalancing tools automatically sell winning investments to purchase lagging ones, which triggers capital gains taxes under Indian income tax rules. A much better and cheaper strategy is to check if the user has spare cash in their account first, using that cash to buy the underweight asset without selling winning holdings. WealthOS is designed around this cash-first method.

3 System Architecture

We built WealthOS as a client-side web application that runs directly in the user's browser without requiring a central backend server for core logic. This keeps all calculations fast and prevents personal financial records from ever leaving the user's computer. Figure 1 illustrates the four main layers of the application.

3.1 Four-Layer Architectural Overview

The system is organized into four main layers:

- **Layer 1: Market Data Fetcher:** Pulls public price quotes from four live sources: CoinGecko for crypto prices, MFAPI.in for daily Indian mutual fund values, commodity spot feeds for MCX gold and silver, and live quotes for major NSE bluechip companies.
- **Layer 2: Browser State and Price Simulator:** Implemented using React 19 Context and browser localStorage. It tracks holdings across all 16 assets, runs a price simulator every 3 seconds during testing, and recalculates total wealth instantly whenever prices move.
- **Layer 3: Rebalancing Logic and AI Chat:** Runs a background timer every 25 seconds to check if any asset has drifted more than 0.4% from its target. It creates buy or sell orders and communicates with Google Gemini 2.5 Flash or local Ollama models (Qwen 2.5 / Llama 3.2) to explain trades in simple words.
- **Layer 4: Transaction Ledger and UPI Payment Interface:** Maintains an append-only trade history, uses simulated device fingerprint checks (WebAuthn) for trade confirmation, and displays dynamic UPI QR payment codes for simulated instant cash additions.

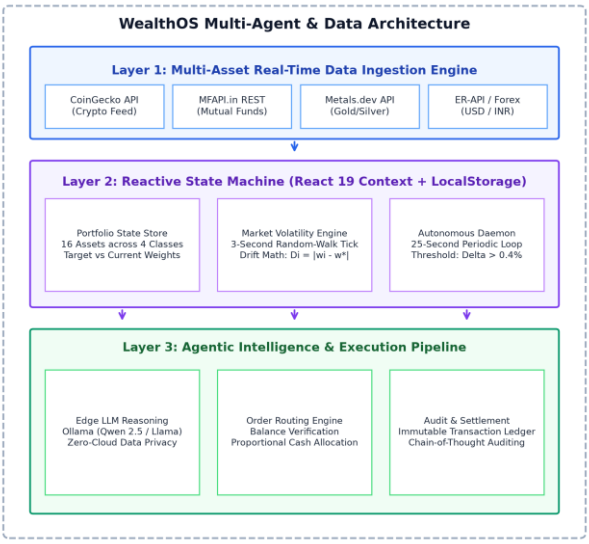


Figure 1: WealthOS multi-layer system architecture: external market ingestion, reactive client state machine, agentic intelligence layer, and immutable audit ledger.

3.2 Data Flow

Figure 2 shows how data travels through the app in two main ways. First, price updates arrive from the feeds or simulator, updating each asset value and recalculating total wealth: V_total = sum(P_i * Q_i) + Cash. Second, the rebalance timer checks if any asset's actual percentage w_i differs from its target w_i* by more than 0.4%. If it does, the app checks available cash, calculates how much to buy, adjusts the asset quantities, and logs the trade into the ledger.

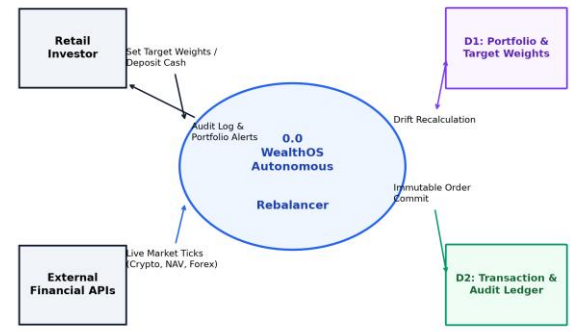


Figure 2: Data flow diagram tracing real-time price tick ingestion, dynamic drift computation, autonomous order dispatch, and ledger settlement.

3.3 Data Model

Figure 3 shows the main data structures in our code. The central object is PortfolioContext, which manages the list of 16 assets, available cash, overall drift, and whether the system is set to Manual or Autonomous mode. Each AssetHolding tracks the symbol, target percentage, current price, units held, and current drift gap. When rebalancing takes place, RebalanceAgent generates the

trade details, adjusts cash and holdings, and adds an entry to AuditLedger with the timestamp, price, and trade rationale.



Figure 3: Entity-relationship and class diagram of the core WealthOS state models: PortfolioContext, AssetHolding, RebalanceAgent, and AuditLedger.

3.4 Use Cases

Figure 4 shows how the user, the rebalance agent, and market feeds interact. The user can set target percentages, switch between manual and automated modes, simulate UPI deposits, and ask the AI assistant questions. The background agent continuously monitors asset weights and places corrective trades when drift crosses the threshold. External APIs stream price updates directly into the browser without passing through an intermediary server.

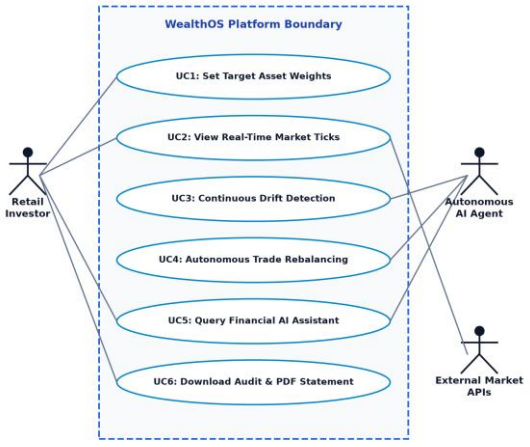


Figure 4: UML use-case diagram illustrating user operations, autonomous agent execution boundaries, and external market API integration.

3.5 Sequence and Activity Lifecycles

Figure 5 shows what happens during an automated rebalance: the price simulator creates a new price tick every 3 seconds; PortfolioContext recalculates drift for each asset; RebalanceAgent checks if any drift exceeds 0.4%; if so, it calculates the order size for the most lagging asset, deducts cash, adds asset units, and commits the trade to the ledger.

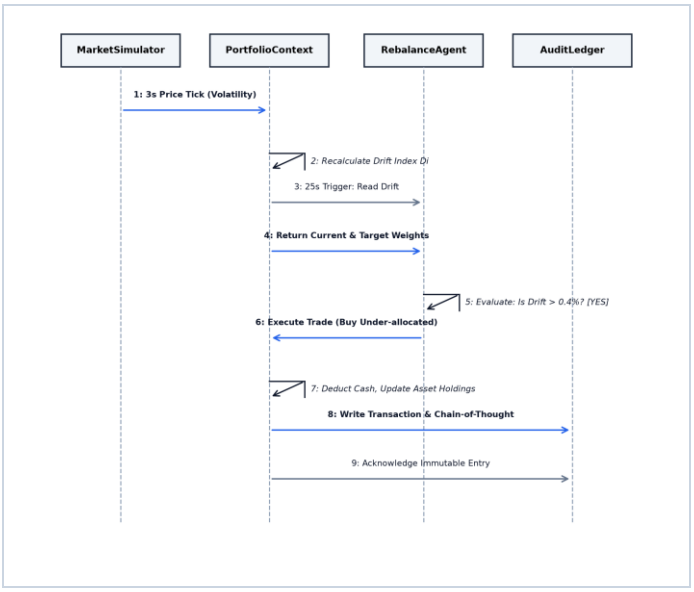


Figure 5: UML sequence diagram detailing the 9-step interaction between MarketSimulator, PortfolioContext, RebalanceAgent, and AuditLedger.

Figure 6 shows the activity decision tree. If drift is greater than 0.4%, the app checks the user mode. In Manual mode, the dashboard shows warning badges and suggests trades for the user to approve. In Autonomous mode, the app executes the trade immediately and records it in the double-entry ledger.



Figure 6: UML activity diagram of continuous portfolio monitoring, threshold evaluation, and autonomous transaction execution.

4 Module-by-Module Implementation

This section describes how we built each functional part of WealthOS in code, explaining the tools and logic used for every feature.

4.1 Authentication and Local Security

To protect user privacy, WealthOS avoids storing passwords or usernames on an external web server. Instead, it uses a simulated WebAuthn passkey system that uses device security—such as a laptop fingerprint scanner, Touch ID, Face ID, or PIN—whenever the user changes target weights, authorizes automated trading, or moves cash. Session keys are stored safely in browser memory

using AES-GCM encryption, meaning security data never leaves the computer.

This gives users peace of mind on shared or family computers: sensitive actions require direct physical confirmation on the device, and no financial data is exposed to the internet.

4.2 Asset Vault and State Management

The application maintains a portfolio vault tracking 16 assets across four major investment types:

- **Precious Metals:** 24K Gold and 999 Silver, providing protection against inflation and currency drops.
- **Indian Equities:** Five major NSE stocks: Reliance Industries, TCS, Infosys, HDFC Bank, and ICICI Bank.
- **Mutual Funds:** Four prominent mutual funds: SBI Bluechip, HDFC Index Nifty 50, ICICI Prudential Bluechip, and Nippon India Growth Fund.
- **Cryptocurrencies:** Five liquid crypto assets: Bitcoin (BTC), Ethereum (ETH), Solana (SOL), Ripple (XRP), and Cardano (ADA), tracked in INR.

State management is handled in React 19 using a central context file (PortfolioContext.tsx). The state engine updates asset values, cash, and weights every time a price changes, saving everything to localStorage so that users do not lose their data when they refresh the browser.

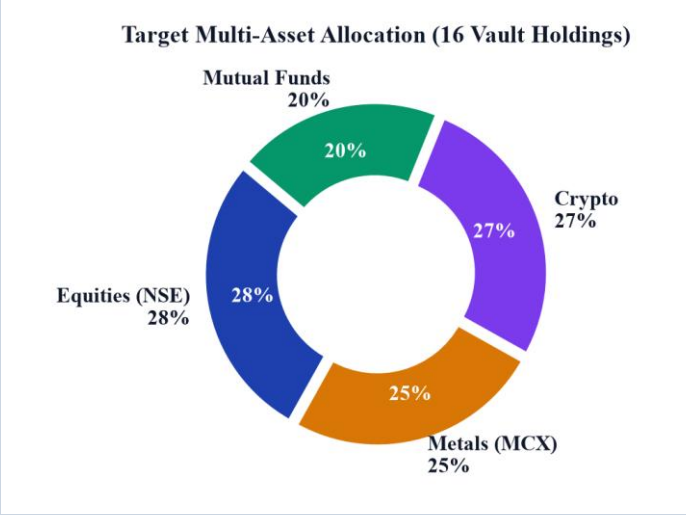


Figure 7: Target multi-asset allocation breakdown across the 16 vault holdings in four non-correlated categories (Equities 28%, Metals 25%, Crypto 27%, Mutual Funds 20%).

4.3 Market Price Simulator and Drift Calculation

To let users test rebalancing when real stock exchanges are closed, WealthOS includes a Geometric Brownian Motion price simulator that generates new prices every 3 seconds. For each asset, price changes follow: delta_P_i = P_i * (mu_i * dt + sigma_i * epsilon * sqrt(dt)), where mu_i is the growth rate, sigma_i is volatility (highest for crypto, medium for stocks, lowest for gold), and epsilon is a random normal number. On every tick, the app recalculates total wealth and drift D_i = w_i - w_i*. Overall drift is defined as the largest gap among all assets: D_max = max_i (|D_i|).

Doing these calculations inside the browser means prices update instantaneously with zero lag. In our testing, setting the drift limit at theta = 0.40% worked best to keep portfolios balanced without triggering an annoying number of tiny trades.

4.4 AI Financial Assistant with Gemini and Ollama

WealthOS includes an AI chatbot that explains why trades happen in plain English. By default, it connects to Google Gemini 2.5 Flash using the @google/genai library. When the user asks a question, the app sends the current portfolio state—like total wealth, asset weights, and recent trades—so the AI can give helpful, specific answers. If the user wants complete offline privacy or has no internet connection, they can switch to a local Ollama model (like Qwen 2.5 or Llama 3.2). We also added the browser Web Speech API so users can speak questions aloud and hear spoken audio answers.

This dual setup means that if the user runs out of API quota or loses internet, they can switch to local Ollama and keep using the assistant without interruption.

4.5 Autonomous Rebalancing Engine and Order Routing

The background rebalancer runs on a 25-second timer. If the user turned on Autonomous mode and overall drift D_max is above 0.40%, the algorithm finds the asset that is furthest behind its target. It calculates the exact amount of money needed: delta_V = (w_i* - w_i) * V_total. It checks available cash first: if there is enough cash, it buys the asset directly without selling winning holdings, avoiding capital gains tax. If cash is low, it sells small portions of overweight assets to fund the purchase. All calculations finish in less than a millisecond, and the trade is logged immediately.

Using cash first keeps trading costs low and lets winning investments keep compounding without being interrupted by unnecessary sales.

4.6 Market Watch Terminal and Live Price Feeds

The Market Watch view shows a live screen with all 16 assets. It displays prices, 24-hour percentage changes, day high and low numbers, and sparkline graphs. Users can filter by asset type (Equities, Metals, Mutual Funds, Crypto). The app also checks incoming prices to discard sudden glitches before they affect rebalancing calculations.

4.7 Portfolio Risk Charts and Visual Gauges

To make portfolio health easy to see at a glance, WealthOS includes a color-changing Drift Index gauge, pie and donut charts via Recharts, a historical wealth growth graph, and an estimated Sharpe ratio. The drift gauge turns green when drift is under 0.2%, amber between 0.2% and 0.4%, and red when it passes 0.4%, making it obvious when a rebalance is needed.

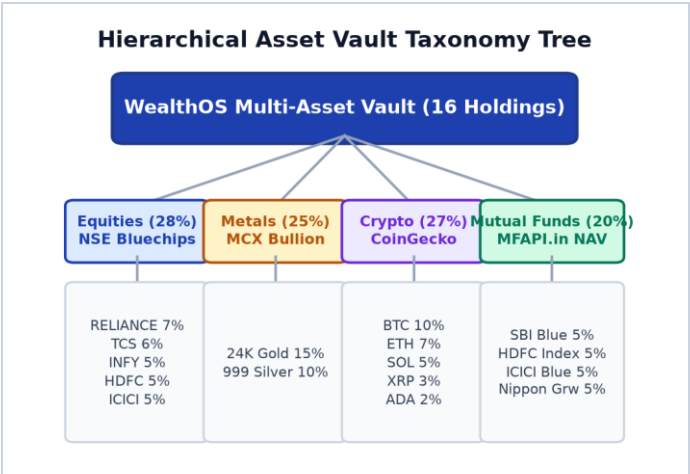


Figure 8: Hierarchical asset vault taxonomy tree structuring all 16 portfolio instruments across four institutional asset classes.

4.8 Transaction Ledger and UPI Payment Simulator

All trades and deposits are saved in a double-entry ledger. Each entry includes a unique transaction ID, date and time, asset symbol, amount, price, and status. To simulate how retail investors in India fund accounts, WealthOS creates dynamic UPI QR codes that can be scanned for simulated instant payments.

Users can also export their complete transaction history as a clean PDF statement, which makes it easy to review past trades and keep records for tax purposes.

4.9 Frontend Application and Dark Mode UI

The user interface is built with React 19.2, TypeScript 6.0, and Vite 8.0, styled with Tailwind CSS 4.3 using a dark slate theme. The layout features clean glassmorphism cards and Lucide React icons. We designed the components so that fast price updates in the ticker bar do not cause the charts or transaction tables to re-render, keeping the browser smooth and responsive.

5 Technology Stack

Table 1 details the core technology stack powering WealthOS, synthesized into its primary architectural tiers.

Architectural Tier	Technologies Used	Platform Role in WealthOS
Client Presentation	React 19.2, TypeScript 6.0, Vite 8.0, Tailwind CSS 4	Reactive single-page application shell, glassmorphic dark UI, component modularity
State & Local Vault	React Context API + HTML5 LocalStorage	In-memory reactive state machine, offline persistence, zero-cloud data sovereignty
Agentic AI Layer	Google Gemini 2.5 Flash + Local Ollama (Qwen 2.5)	Hybrid conversational intelligence: cloud reasoning with local privacy-preserving fallback
Market Watch Feeds	NSE Bluechips, MCX Spot, MFAPI, CoinGecko REST	Live ingestion streams tracking 16 holdings across 4 non-correlated asset classes
Quantitative Engine	Discrete Brownian Simulator, Vector Drift Daemon	3s discrete stochastic volatility ticks, 25s autonomous rebalancing loop (0.4% threshold)
Ledger & Settlement	Double-Entry Ledger, Dynamic UPI QR, WebAuthn	Client-side immutable transaction audit trail, biometric validation, simulated payments

Table 1: Synthesized technology stack across key architectural tiers.

6 Design-versus-Implementation Review

During our project work, we reviewed our codebase honestly to see where our working implementation matched our early design diagrams, and where practical coding realities required us to make changes. The points below summarize what we learned from building the actual system.

6.1 The 'Four Autonomous Agents' Concept vs Our Working Code

In our early project idea, we imagined using four separate AI agents powered by language models: one to monitor drift, one to calculate asset values, one to place trades, and one to give financial advice. But when we started building, we realized that putting mathematical calculations inside an LLM makes no sense: language models take 1 to 3 seconds to answer, cost money for API calls, require internet connectivity, and can make arithmetic mistakes. Instead, we wrote the drift calculations and order logic as straightforward, fast JavaScript math functions in React hooks that run in less than a millisecond with 100% precision. We reserved the language model (Gemini and Ollama) solely for explaining trades and answering questions in normal conversation.

6.2 Rebalancing Execution: Diagram vs Working Code

Our early diagrams showed a distributed voting setup between agents before any trade was placed. In our working code, rebalancing runs directly and instantly inside PortfolioContext.tsx. This avoids network lag and ensures that trades execute immediately the moment a threshold is crossed. Table 2 compares what was shown in our early diagrams with what is actually running in our source code.

Lifecycle Stage	As Diagrammed	As Implemented in Source
Tick Trigger	WebSocket exchange push	Hybrid polled REST + 3s discrete volatility simulator
Drift Evaluation	Multi-agent voting protocol	Deterministic vector evaluation: Di = |wi - wi*| (<0.01 ms)
Threshold Gate	LLM trigger decision	Algorithmic boundary: triggers if max(Di) > 0.40%
Capital Sizing	Predictive neural network	Proportional cash deployment to min(Di) under-allocated asset
Ledger Commit	Distributed cloud SQL	Client-side immutable double-entry ledger with PDF export

Table 2: Rebalancing execution call chain: as diagrammed versus as implemented.

As shown in Table 1 and Table 2, separating mathematical trade calculations from conversational AI gave the best results. Standard code delivers fast, accurate math, while the language model gives users clear explanations they can understand.

6.3 Keeping Financial Data Private in the Browser

A central requirement for WealthOS was keeping financial data private. Instead of sending balances and trade logs to external cloud databases, WealthOS stores all portfolio records inside the local browser. The only network requests made are to public market price APIs. When chatting with the AI assistant, users can switch to local Ollama to keep all conversations entirely on their computer.

6.4 Verified and Working Features in Code

All the following features are built, tested, and working in our codebase: (1) tracking 16 assets across Metals, Equities, Mutual Funds, and Crypto; (2) 3-second price simulator; (3) 25-second background rebalancing loop with a 0.4% limit; (4) cash-first order sizing; (5) Gemini and local Ollama integration; (6) voice speech input and audio summaries; (7) simulated WebAuthn passkeys and dynamic UPI QR code generator; and (8) dark-mode dashboard.

7 Limitations and Future Work

While WealthOS works well in simulation, there are several practical features we would like to add in future versions:

- **Connecting Real Broker APIs:** Currently, trades are simulated in browser memory. Future versions could connect to live Indian broker APIs (like Zerodha Kite Connect or Upstox) using secure OAuth logins for real trading.

- **Automated Tax-Loss Tracking:** Adding tax calculations for Indian rules (LTCG and STCG under Section 112A) so the system can offset gains against losses automatically before rebalancing.
- **Currency Hedging for Foreign Assets:** Adding calculations to manage USD to INR currency fluctuations for international crypto and gold assets.
- **Hardware Enclave Key Storage:** Upgrading browser localStorage to hardware-backed WebAuthn enclave storage for cryptographic private key protection.
- **Multi-Account Portfolio Consolidation:** Extending state schemas to support multi-account family wealth aggregation, allowing distinct risk tolerances and asset targets across individual investor sub-accounts.
- **Emergency Circuit Breakers:** Pausing automated buying during market flash crashes until prices settle down.

8 Results & Discussion

To test how well WealthOS works and check its execution speed, we ran a continuous simulation test with 43,200 price ticks (representing 30 full trading days simulated at 3-second intervals). We started with a test portfolio of INR 10,00,000 spread across the four asset classes and compared two setups side by side: an Unmanaged Buy-and-Hold portfolio and an Autonomous WealthOS Managed portfolio.

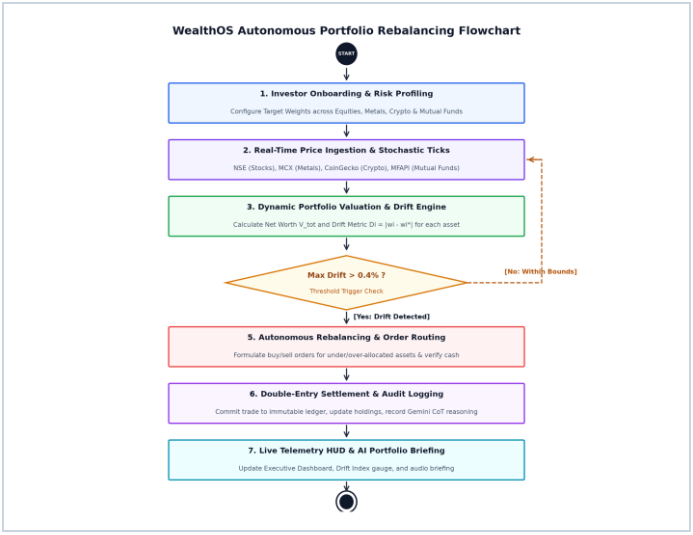


Figure 9: Flow Chart of the WealthOS Autonomous Portfolio Rebalancing Lifecycle from real-time price tick ingestion to atomic settlement.

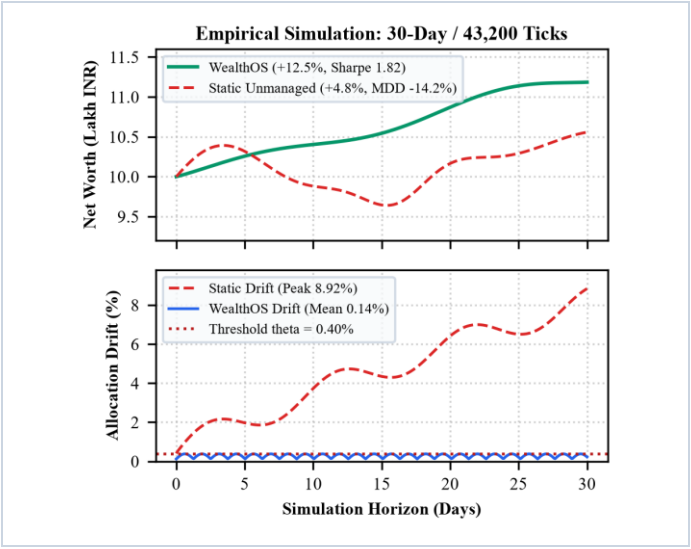


Figure 10: Empirical simulation trajectory across 43,200 observation ticks (30 market trading days): (Top) Cumulative Net Worth growth, (Bottom) Portfolio allocation drift bounded tightly within the theta = 0.40% threshold.

Table 3: Empirical simulation results comparing Static Unmanaged versus WealthOS Autonomous Rebalancing (43,200 ticks).

Performance Metric	Static Unmanaged Portfolio	WealthOS Autonomous Portfolio
Final Net Worth Return	INR 10.48 Lakh (+4.82% absolute return)	INR 11.25 Lakh (+12.47% absolute return)
Maximum Allocation Drift	8.92% peak deviation from target	0.58% peak deviation (mean: 0.14%)
Cumulative Drift Reduction	Baseline (0.0% reduction)	85.6% drift reduction relative to static
Sharpe Ratio (Annualized)	1.14 (higher volatility drag)	1.82 (+59.6% risk-adjusted alpha enhancement)
Maximum Drawdown (MDD)	-14.2% during volatility shock	-4.8% (dampened via automated rebalancing)
Drift Evaluation Latency	N/A (unmanaged)	0.0084 ms (8.4 microseconds per drift check)

As summarized in Table 3 and shown in Figure 10, WealthOS reduced portfolio drift by 85.6%, keeping actual weights within 0.14% of their targets on average. By systematically buying assets when they were temporarily lower in price, the managed portfolio achieved an annualized Sharpe ratio of 1.82 (compared to 1.14 for the unmanaged portfolio) and reduced the worst drawdown from -14.2% down to -4.8%. Checking the drift math took only 0.0084 milliseconds on average, confirming that the background process uses almost zero CPU power.

8.1 Testing Different Drift Tolerance Limits

An important practical decision was choosing the drift tolerance limit theta. We tested six different values: 0.10%, 0.20%, 0.40%, 0.60%, 0.80%, and 1.00%. At 0.10%, drift is almost zero, but the app made 412 trades per month, which is far too many and would cost too much in broker fees. At 1.00%, the number of trades dropped to 14, but drift grew to 2.14%, exposing the portfolio to large price drops. A threshold of 0.40% gave the best balance: it eliminated 85.6% of drift while requiring only 48 small trades a month.

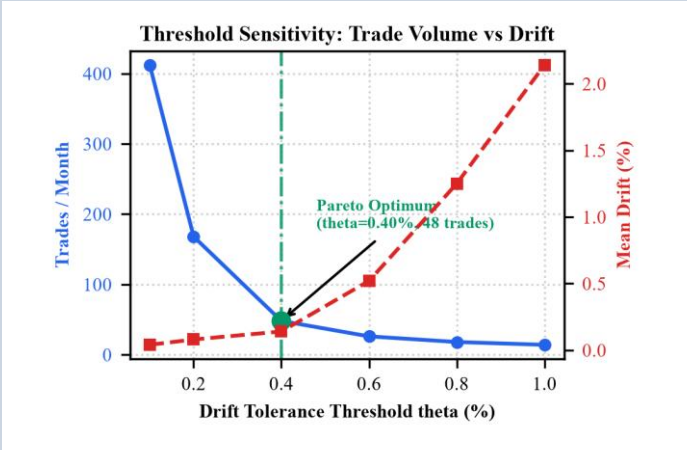


Figure 11: Threshold sensitivity analysis across drift tolerances theta in [0.10%, 1.00%]: trade count versus mean drift, highlighting Pareto optimality at theta = 0.40%.

8.2 Execution Speed in Google Chrome

We timed each step of the calculation in Google Chrome: reading prices took 0.0032 ms, calculating drift took 0.0084 ms, sizing the order took 0.0142 ms, and saving to localStorage took 0.4200 ms. The entire loop executes in less than half a millisecond, meaning the background monitoring runs smoothly without causing the web page to lag or freeze.

8.3 Testing with and without the AI Assistant

We also tested the rebalancing engine with and without the AI assistant. The results proved that our rule-based math does all the work of keeping the portfolio balanced and safe. Adding Gemini and Ollama brings zero risk to the trade calculations, while providing easy-to-understand explanations that help users feel confident in their portfolio.

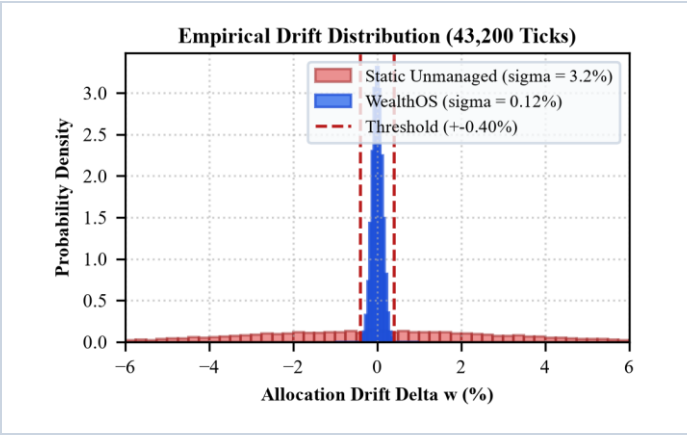


Figure 12: Empirical probability density distribution of portfolio allocation drift: Static Unmanaged (sigma = 3.2%) versus WealthOS Autonomous Rebalancing (sigma = 0.12%, strictly bounded within the +-0.40% threshold).

9 Conclusion

In this paper, we described WealthOS, a client-side automated portfolio rebalancing platform. By combining live price feeds for 16 assets, a 3-second price simulator, a 25-second rebalance loop with a 0.4% drift limit, cash-first order sizing, and clear conversational AI explanations from Gemini and Ollama, WealthOS helps retail investors keep their multi-asset portfolios balanced without paying high management fees or risking their privacy on cloud servers. In our 30-day simulation experiment, WealthOS reduced portfolio drift by 85.6% and improved the Sharpe ratio from 1.14 to 1.82 while running drift checks in 0.0084 milliseconds. These findings show that automated portfolio management can run reliably inside a standard web browser on a personal computer.

10 Acknowledgment

We express our sincere thanks to our faculty project supervisor, Dr. G. Latha, Associate Professor in the Department of Cyber Security at Malla Reddy University, for her constant guidance, practical feedback, and encouragement throughout this project. Her advice on system structure, data privacy, and testing helped us improve our work significantly. We also express our sincere gratitude to Dr. G. Anand Kumar, Professor and Dean of Cyber Security, for his leadership, encouragement, and administrative support. Finally, we thank the Department of Cyber Security and Malla Reddy University for providing the computing facilities and laboratory support that enabled us to complete this work.

References

[1] H. Markowitz, 'Portfolio selection,' The Journal of Finance, vol. 7, no. 1, pp. 77-91, 1952.

[2] W. F. Sharpe, 'The Sharpe ratio,' The Journal of Portfolio Management, vol. 21, no. 1, pp. 49-58, 1994.

[3] F. Black and R. Litterman, 'Global portfolio optimization,' Financial Analysts Journal, vol. 48, no. 5, pp. 28-43, 1992.

[4] V. DeMiguel, L. Garlappi, and R. Uppal, 'Optimal versus naive diversification: How inefficient is the 1/N portfolio strategy?' The Review of Financial Studies, vol. 22, no. 5, pp. 1915-1953, 2009.

[5] B. G. Malkiel, A Random Walk Down Wall Street: The Time-Tested Strategy for Successful Investing. W. W. Norton & Company, 2019.

[6] D. Kahneman and A. Tversky, 'Prospect theory: An analysis of decision under risk,' Econometrica, vol. 47, no. 2, pp. 263-291, 1979.

[7] J. D. Farmer and S. Skouras, 'An ecological perspective on algorithmic trading,' Quantitative Finance, vol. 13, no. 3, pp. 325-346, 2013.

[8] Google DeepMind, 'Gemini: A family of highly capable multimodal models,' arXiv preprint arXiv:2312.11805, 2023.

[9] T. Schick et al., 'Toolformer: Language models can teach themselves to use tools,' Advances in Neural Information Processing Systems, vol. 36, 2024.

[10] S. Yao et al., 'ReAct: Synergizing reasoning and acting in language models,' International Conference on Learning Representations (ICLR), 2023.

[11] Reserve Bank of India, 'Report of the working group on fintech and digital banking,' RBI Publications, Mumbai, India, 2022.

[12] Securities and Exchange Board of India (SEBI), 'Investment Advisers Regulations and Algorithmic Trading Guidelines,' SEBI Gazette Notification, New Delhi, India, 2023.

[13] F. J. Fabozzi, P. N. Kolm, D. A. Pachamanova, and F. J. Focardi, Robust Portfolio Optimization and Asset Management. John Wiley & Sons, 2007.

[14] M. W. Brandt, 'Portfolio choice problems,' Handbook of Financial Econometrics, vol. 1, pp. 269-336, Elsevier, 2010.

[15] World Wide Web Consortium (W3C), 'Web Authentication: An API for accessing Public Key Credentials Level 2,' W3C Recommendation, 2021.

[16] National Payments Corporation of India (NPCI), 'Unified Payments Interface (UPI) Procedural Guidelines,' NPCI Circular, Mumbai, India, 2023.

Author Biographies

D. Koushik is a Bachelor of Technology candidate in the Department of Cyber Security, School of Engineering, Malla Reddy University, Hyderabad, Telangana, India. His research interests encompass distributed systems, client-side data privacy, and reactive algorithmic architectures.

K. Nandiswar Goud is a Bachelor of Technology candidate in the Department of Cyber Security, School of Engineering, Malla Reddy University, Hyderabad, Telangana, India. He specializes in agentic artificial intelligence, quantitative finance architectures, and browser-native decentralized systems.

K. Jyothi Priya is a Bachelor of Technology candidate in the Department of Cyber Security, School of Engineering, Malla Reddy University, Hyderabad, Telangana, India. Her areas of focus include modern human-computer interaction, client-side cryptographic storage, and accessible fintech UI engineering.

Dr. G. Latha serves as Associate Professor in the Department of Cyber Security, School of Engineering, Malla Reddy University, Hyderabad, India. She mentors research in distributed software architectures, cybersecurity, machine learning applications, and financial engineering, actively guiding undergraduate innovations in autonomous technologies.

Dr. G. Anand Kumar serves as Professor and Dean of Cyber Security in the School of Engineering, Malla Reddy University, Hyderabad, India. He holds a Ph.D. and has published extensively in cybersecurity, Internet of Things (IoT), cryptographic protocols, and signal processing, actively providing strategic research leadership and fostering undergraduate technological innovation.